

US-009 — Implement Standard API Response

Personal Finance & Money Flow Tracker — Detailed implementation and learning guide for freshers

US-009 — Implement Standard API Response

Personal Finance & Money Flow Tracker

Estimate: 4 hours

Technology: Google Apps Script, JavaScript, Google Sheets backend

Audience: Freshers and new contributors

1. Purpose

US-009 establishes one standard JSON response contract for the Personal Finance Tracker API.

Success response

```
{
  "success": true,
  "message": "Request successful",
  "data": {}
}
```

Error response

```
{
  "success": false,
  "error": {
    "code": "INVALID_REQUEST",
    "message": "Request body is required"
  }
}
```

This story standardizes response formatting. Transaction CRUD, validation business rules, master-data retrieval, and balance calculations remain outside this story.

2. Architecture

```
Browser / Postman
|
v
Google Apps Script Web App
|
v
Code.gs
doGet / doPost
|
v
Api.gs
parse + route
|
v
Service Layer
|
v
ResponseUtil.gs
+----- success()
|
+----- error()
|
v
toJsonResponse()
```

|
v
application/json

The key design principle is to centralize response creation in `ResponseUtil.gs`.

3. US-009 Requirements

- Implement success response.
- Implement error response.
- Add `success`.
- Add `message`.
- Add `data`.
- Add `error`.
- Add `code`.
- Standardize HTTP/API handling.

4. Response Contract

Success

Field Type Purpose

--- --- ---

success Boolean Indicates success

message String Human-readable result

data Object/Array Response payload

Example:

```
{
  "success": true,
  "message": "Transaction created successfully",
  "data": {
    "transactionId": "TXN-001",
    "amount": 2500
  }
}
```

Error

Field Type Purpose

--- --- ---

success Boolean Indicates failure

error Object Error details

error.code String Machine-readable error identifier

error.message String Human-readable description

Example:

```
{
  "success": false,
  "error": {
```

```

    "code": "INVALID_REQUEST",
    "message": "Request body is required"
  }
}

```

5. Files

```

PersonalFinanceTracker-Backend
|
+-- Code.gs
+-- Api.gs
+-- TransactionService.gs
+-- AccountService.gs
+-- BalanceService.gs
+-- ValidationService.gs
+-- SheetRepository.gs
+-- ResponseUtil.gs
+-- Config.gs
+-- ResponseUtilTest.gs

```

US-009 updates `ResponseUtil.gs`, verifies integration with `Code.gs` and `Api.gs`, and adds `ResponseUtilTest.gs`.

6. Code.gs

```

function doGet(e) {
  return Api.handleGetRequest(e);
}

function doPost(e) {
  return Api.handlePostRequest(e);
}

```

`doGet(e)` is the Google Apps Script entry point for GET requests. `doPost(e)` is the entry point for POST requests. Keeping these functions thin prevents business logic from being mixed with HTTP entry-point code.

7. ResponseUtil.gs — Complete Implementation

```

const ResponseUtil = {

  /**
   * Creates a standardized successful API response.
   *
   * Response format:
   *
   * {
   *   "success": true,
   *   "message": "Request successful",
   *   "data": {}
   * }
   */
  success: function(data, message) {

    const response = {
      success: true,
      message: message || 'Request successful',
      data: data || {}
    };

    return this.toJsonResponse(response);
  },

  /**
   * Creates a standardized API error response.
   *
   * Response format:
   *
   * {
   *   "success": false,
   *   "error": {
   *     "code": "ERROR_CODE",
   *     "message": "Error message"
   *   }
   * }
   */
}

```

```

    *   }
    */
    error: function(code, message) {

        const response = {
            success: false,
            error: {
                code: code || 'SERVER_ERROR',
                message: message || 'An unexpected error occurred'
            }
        };

        return this.toJsonResponse(response);
    },

    /**
     * Converts a JavaScript object into
     * a Google Apps Script JSON response.
     */
    toJsonResponse: function(response) {

        return ContentService
            .createTextOutput(JSON.stringify(response))
            .setMimeType(ContentService.MimeType.JSON);
    }

};

---
```

8. Understanding success()

```

ResponseUtil.success(
    {
        transactionId: 'TXN-001'
    },
    'Transaction created successfully'
);
```

Produces:

```

{
  "success": true,
  "message": "Transaction created successfully",
  "data": {
    "transactionId": "TXN-001"
  }
}
```

`message || 'Request successful'` supplies a default message. `data || {}` supplies an empty object when no data is provided.

9. Understanding error()

```

ResponseUtil.error(
    'INVALID_REQUEST',
    'Request body is required'
);
```

Produces:

```

{
  "success": false,
  "error": {
    "code": "INVALID_REQUEST",
    "message": "Request body is required"
  }
}
```

The error code is intended for application logic. The message is intended to be human-readable.

10. Understanding toJsonResponse()

```
toJsonResponse: function(response) {
    return ContentService
        .createTextOutput(JSON.stringify(response))
        .setMimeType(ContentService.MimeType.JSON);
}
```

Three operations happen:

1. `JSON.stringify()` converts a JavaScript object into a JSON string.
2. `createTextOutput()` creates the Apps Script response body.
3. `setMimeType(ContentService.MimeType.JSON)` identifies the response as JSON.

This prevents every endpoint from duplicating the same HTTP/JSON response code.

11. Standard Error Codes

Code Meaning

INVALID_REQUEST Request structure is invalid

INVALID_JSON Request body is not valid JSON

UNKNOWN_ACTION Requested action is unsupported

NOT_IMPLEMENTED Route exists but business operation is not implemented yet

SERVER_ERROR Unexpected server-side failure

Future service stories can add domain-specific codes such as `ACCOUNT\_NOT\_FOUND`, `CATEGORY\_NOT\_FOUND`, and `TRANSACTION\_NOT\_FOUND`.

12. Api.gs Integration

The API layer should return responses through `ResponseUtil`.

Examples:

```
return ResponseUtil.success(
    {
        message: CONFIG.APP_NAME + ' API is running'
    },
    'Request successful'
);

return ResponseUtil.error(
    'INVALID_REQUEST',
    'Request body is required'
);

return ResponseUtil.error(
    'UNKNOWN_ACTION',
    'Unsupported GET action: ' + action
);
```

Avoid manually creating JSON responses inside `Api.gs`. Centralization is the purpose of `ResponseUtil`.

13. GET Flow

For:

```
GET /exec
```

the flow is:

```
Browser
  |
  v
doGet()
  |
  v
Api.handleGetRequest()
  |
  v
ResponseUtil.success()
  |
  v
JSON
```

Expected:

```
{
  "success": true,
  "message": "Request successful",
  "data": {
    "message": "Personal Finance Tracker API is running"
  }
}
```

14. GET Error Flow

Request:

```
GET /exec?action=GET_TRANSACTION
```

without a transaction ID.

Expected:

```
{
  "success": false,
  "error": {
    "code": "INVALID_REQUEST",
    "message": "transactionId is required"
  }
}
```

15. POST Flow

Example:

```
POST /exec
Content-Type: application/json
```

Body:

```
{
  "action": "CREATE_TRANSACTION",
  "data": {
    "amount": 2500
  }
}
```

The API parses JSON, identifies the action, routes it, and returns a standardized response.

At US-009 stage, actual transaction creation is intentionally not implemented.

16. ResponseUtilTest.gs — Complete Test Script

```
function testSuccessResponse() {  
    const response = ResponseUtil.success(  
        {  
            transactionId: 'TXN-001',  
            amount: 2500  
        },  
        'Transaction created successfully'  
    );  
    Logger.log(response.getContent());  
}  
  
function testDefaultSuccessResponse() {  
    const response = ResponseUtil.success();  
    Logger.log(response.getContent());  
}  
  
function testErrorResponse() {  
    const response = ResponseUtil.error(  
        'INVALID_REQUEST',  
        'Request body is required'  
    );  
    Logger.log(response.getContent());  
}  
  
function testDefaultErrorResponse() {  
    const response = ResponseUtil.error();  
    Logger.log(response.getContent());  
}
```

17. Test Results

Test 1 — Success response

Run `testSuccessResponse()`.

Expected:

```
{  
  "success": true,  
  "message": "Transaction created successfully",  
  "data": {  
    "transactionId": "TXN-001",  
    "amount": 2500  
  }  
}
```

Test 2 — Default success

Run `testDefaultSuccessResponse()`.

Expected:

```
{  
  "success": true,  
  "message": "Request successful",  
  "data": {}  
}
```

Test 3 — Error response

Run `testErrorResponse()`.

Expected:

```
{
  "success": false,
  "error": {
    "code": "INVALID_REQUEST",
    "message": "Request body is required"
  }
}
```

Test 4 — Default error

Run `testDefaultErrorResponse()`.

Expected:

```
{
  "success": false,
  "error": {
    "code": "SERVER_ERROR",
    "message": "An unexpected error occurred"
  }
}
```

18. API Integration Tests

Health check

GET /exec

Expected success response.

Unknown GET action

GET /exec?action=ABC

Expected:

```
{
  "success": false,
  "error": {
    "code": "UNKNOWN_ACTION",
    "message": "Unsupported GET action: ABC"
  }
}
```

Missing transaction ID

GET /exec?action=GET_TRANSACTION

Expected:

```
{
  "success": false,
  "error": {
    "code": "INVALID_REQUEST",
    "message": "transactionId is required"
  }
}
```

Invalid JSON

POST body:

```
{ invalid json }
```

Expected:

```
{
  "success": false,
  "error": {
    "code": "INVALID_JSON",
    "message": "Request body must contain valid JSON"
  }
}
```

Missing action

```
{
  "data": {}
}
```

Expected:

```
{
  "success": false,
  "error": {
    "code": "INVALID_REQUEST",
    "message": "Action is required"
  }
}
```

Unknown POST action

```
{
  "action": "ABC"
}
```

Expected:

```
{
  "success": false,
  "error": {
    "code": "UNKNOWN_ACTION",
    "message": "Unsupported POST action: ABC"
  }
}
```

19. Postman Test

POST URL:

https://script.google.com/macros/s/AKfycbx2fZsDgM3RCFVksQhM1ai4kr4kyc7AbTfWUP1QWdwZhQwmhdqX_ZG03WDQhsIm5IyC/exec

Header:

Content-Type: application/json

Body:

```
{
  "action": "CREATE_TRANSACTION",
  "data": {
    "transactionDate": "07-09-2026",
    "transactionType": "EXPENSE",
    "amount": 2500,
    "categoryId": "CAT001",
    "paymentMethod": "BANK",
    "paidFromAccountId": "ACC001",
    "notes": "Dinner"
  }
}
```

Expected at this stage:

```
{
  "success": false,
  "error": {
    "code": "NOT_IMPLEMENTED",
    "message": "CREATE_TRANSACTION is not implemented yet"
  }
}
```

```
}  
}
```

`NOT\_IMPLEMENTED` is correct here because US-009 standardizes responses and does not implement transaction CRUD.

20. Before vs After US-009

Before

Response creation could be repeated across endpoints.

After

```
Endpoint  
|  
+--> ResponseUtil.success()  
|  
+--> ResponseUtil.error()  
      |  
      v  
    toJsonResponse()  
      |  
      v  
    application/json
```

Benefits:

- Consistent API contract.
- Less duplicate code.
- Easier frontend integration.
- Easier testing.
- Easier maintenance.
- Centralized JSON MIME handling.
- Consistent error codes.
- Easier onboarding.

21. Beginner Concepts

API Contract

An API contract defines what the backend promises to return.

Success:

```
success  
message  
data
```

Error:

```
success  
error.code  
error.message
```

Machine-readable vs human-readable

`error.code` is for application logic:

```
ACCOUNT_NOT_FOUND
```

`error.message` is for people:

```
The selected account could not be found.
```

JSON

JSON is a common format for transferring structured data between clients and servers.

GET vs POST

GET is generally used to retrieve information.

POST is generally used to submit information.

Separation of concerns

`Code.gs` handles entry points, `Api.gs` handles request/routing logic, and `ResponseUtil.gs` handles response formatting. Keeping these responsibilities separate makes the project easier to understand and maintain.

22. Deployment

After saving changes:

1. Open Deploy.
2. Select Manage deployments.
3. Edit the Web App deployment.
4. Create/select the new version.
5. Deploy.
6. Test the `/exec` URL again.

The deployed Web App must use the updated version before external tests can verify the new code.

23. Git Workflow

```
git checkout develop
git pull origin develop
git checkout -b feature/US-009-standard-api-response
```

After implementation:

```
git status
git add .
git commit -m "feat: implement standard API responses"
git push -u origin feature/US-009-standard-api-response
```

PR:

```
feature/US-009-standard-api-response
|
v
develop
```

Suggested PR title:

```
feat: implement standard API response handling
```

24. Definition of Done

Implementation

- [x] Success response implemented.
- [x] Error response implemented.
- [x] `success` standardized.
- [x] `message` standardized.
- [x] `data` standardized.
- [x] `error` standardized.
- [x] `error.code` standardized.
- [x] `error.message` standardized.
- [x] JSON response creation centralized.
- [x] JSON MIME type configured.

Integration

- [x] `doGet()` uses API layer.
- [x] `doPost()` uses API layer.
- [x] API errors use `ResponseUtil`.
- [x] Existing US-008 routing remains compatible.

Testing

- [x] Success response tested.
- [x] Default success tested.
- [x] Error response tested.
- [x] Default error tested.
- [x] Invalid JSON tested.
- [x] Missing body tested.
- [x] Missing action tested.
- [x] Unknown action tested.
- [x] GET tested.
- [x] POST tested.
- [x] Web App redeployed and verified.

25. Scope Boundaries

US-009 does not implement:

Transaction CRUD
Account CRUD
Category CRUD
Balance calculations
Transaction validation rules
Master data retrieval
Frontend integration

Authentication
Authorization

These belong to other stories/layers.

26. Final Learning Summary

US-009 creates the response foundation for the Personal Finance Tracker backend.

Core success API:

```
ResponseUtil.success(data, message);
```

Core error API:

```
ResponseUtil.error(code, message);
```

Both use:

```
toJsonResponse(response);
```

Final success contract:

```
{
  "success": true,
  "message": "...",
  "data": {}
}
```

Final error contract:

```
{
  "success": false,
  "error": {
    "code": "...",
    "message": "..."
  }
}
```

The result is a predictable API contract that future backend services and the frontend can consume consistently.